UNIVERSIDADE
LUSÓFONA

Licenciatura em Engenharia Informática

Project I

Beta Breaker

Project Report

Miguel Leitão Cardoso

miguelcardoso499@proton.me

Advisor: Advisor Name

External Entity: N/A

Department of Computer Science Engineering and Information Systems

Universidade Lusófona, Centro Universitário do Porto (CUP)

2025/2026

www.ulusofona.pt

Copyright

Acknowledgments

Resumo

Palavras-chave: escalar; ginásio; gamificação; beta; mobile

Abstract

Keywords: climbing; gym; gamification; beta; mobile

Contents

List of Figures

List of Tables

List of Acronyms

API Application Programming Interface

B2B2C Business-to-Business-to-Consumer

CORS Cross-Origin Resource Sharing

CSV Comma-Separated Values

GDPR General Data Protection Regulation

HTTPS Hypertext Transfer Protocol Secure

IAP In-App Purchase

IDE Integrated Development Environment

MVP Minimum Viable Product

NFC Near Field Communication

PDF Portable Document Format

PII Personally Identifiable Information

QR Quick Response (code)

RLS Row Level Security

SRS Software Requirements Specification

SWOT Strengths, Weaknesses, Opportunities, Threats

TLS Transport Layer Security

TOC Table of Contents

UGC User-Generated Content

1 Introduction

1.1 Project Overview

The project consists of developing a mobile application to support the digitalization and gamification of indoor climbing gyms. The application will allow climbers to view information about each climbing route, such as grade, wall location, and skills emphasized (e.g., finger strength, technique, dynamic movement, footwork, endurance). Users will also be able to submit beta videos, leave comments, and review routes to help others progress.

To enhance motivation, the app will include gamification features such as achievements (badges for completing challenges, milestone ascents, or gym-specific objectives) and leaderboards (ranking climbers based on performance, number of routes completed, or difficulty level). Leaderboards will be complemented with prizes to reward top performers and stimulate continuous participation.

Additionally, the app will allow climbers to provide structured feedback to route setters, enabling gyms to collect insights about route quality, difficulty perception, and overall user experience.

The main goal is to create a community-driven platform that increases engagement with climbing gyms, motivates climbers through interactive features, and provides a structured way to analyze training and route styles.

1.2 Problem Statement

Indoor climbers often lack a unified way to access up-to-date route information, beta videos, and meaningful progression tracking inside their gyms. Gyms also need a simple method to manage their routes and receive feedback from the climbers. Beta Breaker addresses these issues by providing climbers with clear route information, shared beta, achievements, and leaderboards that encourage consistent participation, while giving gyms a modern digital layer to manage their setting and community.

1.3 High-Level Narrative

Beta Breaker is a platform that connects climbers and gyms around shared climbing experiences. It helps gyms publish and manage their indoor routes and community, while giving climbers a simple way to discover, log, and learn from their climbs. The product emphasizes fairness, meaningful feedback, and clear insight into progression, supporting both everyday sessions and gym-led initiatives such as seasons and events.

1.4 Vision

Product name: Beta Breaker

Vision statement. Create the go-to mobile platform that connects climbers and gyms through route discovery, beta sharing, and light-weight gamification. Beta Breaker motivates climbers to progress, helps gyms understand their community, and gives route setters actionable feedback to improve route quality.

Problem. Indoor climbing is highly social, but knowledge about routes (beta, perceived difficulty, style) is fragmented across word-of-mouth and social media. Gyms lack structured, privacy-respectful ways to engage customers and collect feedback; climbers lack a single place to log, learn, and stay motivated.

Solution. A mobile app (iOS/Android) where climbers can browse routes, watch/upload beta videos, leave comments and ratings, and participate in challenges, achievements, and leaderboards. Gyms and route setters receive summarized feedback and simple analytics on route reception and difficulty perception.

Primary users. (1) Climbers who want to progress and engage with the community; (2) Route setters / gyms who want structured feedback and engagement tools.

1.5 Objectives

Business Objectives (Project Charter)

The business objectives of this project are:

- Support climbing gyms in offering a modern digital service that complements their physical routes.
- Increase customer engagement and retention by gamifying the climbing experience.
- Provide climbers with tools to track progress, exchange knowledge, and connect with the community.
- Motivate climbers through gamification features such as achievements, challenges, leaderboards, and prizes.
- Strengthen the visibility of climbing gyms through digital integration and user-generated content.
- Provide route setters with valuable feedback from climbers, helping them adjust and improve future routes.

Business Goals (SRS)

- Increase climber engagement and retention through scan-to-log flows and rich feedback.
- Provide gyms with verifiable leaderboards, operational visibility, and simple billing.
- Enable fair progression by crowd-sourced tags and optional video verification.
- Monetize via Pro subscriptions (consumers) and per-active-user billing (gyms).

1.6 Decomposition into Sub-Problems

- **Identity & Accounts:** registration, profile, privacy and safety.
- **Gyms & Areas:** gym directory, sectors, social links.
- **Routes Catalog:** route model, QR/NFC IDs, user-driven tags and feedback.
- **Logging & Sessions:** quick logs, session start/end, summaries.
- **Analytics:** grade conversions, personalized suggestions (Pro).
- **Gamification:** badges, streaks, optional rank model.
- **Social:** leaderboards, feedback, reporting, video verification policy.
- **Admin Web:** audit logs and operations.
- **Monetization:** Pro subscriptions and gym billing console.

1.7 Project Scope

1.7.1 In Scope (MVP)

- **Route catalog:** Browse/search routes by grade, wall/sector, style/tags (e.g., finger strength, dynamic movement, footwork, endurance).
- **Beta and reviews:** View, upload, and report beta videos (basic moderation tools); leave comments and 1–5 rating plus perceived grade.
- **Gamification:** Achievements (e.g., first of grade, weekly streaks), simple leaderboards (per gym, per period), and optional prizes managed by gyms.
- **Feedback to setters:** Structured form per route (quality, grade perception, style markers); aggregated summaries and trends for gyms.
- **Accounts & privacy:** Email/social login, profile, basic notification settings, GDPR-aligned consent screens.
- **Admin (lightweight):** Gym/route management and review moderation (web or mobile admin views).

1.7.2 Out of Scope (for now)

- Payments, in-app purchases, and complex prize logistics.
- Advanced training plans, wearables integration, or full performance analytics.
- Outdoor crag support.

1.7.3 Key Deliverables

- Lo-fi & Hi-fi prototypes with core user flows (browse, beta, feedback, achievements).
- Functional MVP implementing the features listed in "In Scope (MVP)".
- Pilot deployment with seeded routes for at least one gym and a small user group.
- Documentation (requirements, architecture, test plan, project report) and a final presentation.

1.7.4 Quality Attributes

- **Usability:** Clear navigation, short paths to core actions (watch beta, log feedback).
- **Performance:** Route list loads in < 2 s on average Wi-Fi; videos stream with adaptive quality.
- **Security & privacy:** Least-privilege data access; user control over content and account data.
- **Maintainability:** Modular architecture; documented APIs and data models.

1.8 Glossary of Terms

Route/Problem
: A climb in the gym (boulder, top-rope, lead).

QR/NFC Tag
: Physical tag to open the route page in-app.

Beta
: Strategy or movement sequence to complete a route.

Verification Video
: Video evidence attached to a send for leaderboard validity.

Pro Tier
: Paid consumer plan with analytics, unlimited beta views, and 3 profile badges.

1.9 Document Structure

This report is organized as follows. Chapter 1 presents the context, motivation, identified problem, and the project objectives. Chapter 2 discusses the relevance and feasibility of the

solution, including a review of existing solutions, the proposed added value, and the identified business opportunity. Chapter 3 focuses on specification and modelling, covering functional and non-functional requirements, the data model (ER diagram), and the interface prototypes. Chapter 4 describes the developed solution, including its architecture, technologies, and components. Chapter 5 covers the testing and validation approach, tools, and coverage. Chapter 6 presents the development methodology, work plan, and progress analysis. Chapter 7 reports test results and requirements compliance. Chapter 8 summarizes the conclusions and outlines future work.

2 Relevance and Feasibility

2.1 Relevance

Indoor climbing gyms continue to grow and increasingly rely on community engagement and fresh route setting to retain members. However, route information, style perception, and beta knowledge are still largely informal and fragmented. A platform that is tightly connected to the physical gym environment and its route lifecycle can improve the experience for climbers while providing gyms with structured feedback and actionable insights.

2.2 Feasibility

The project is designed as an MVP-first solution with an incremental delivery strategy. The scope is controlled to avoid complex payment flows and heavy analytics at this stage, focusing instead on core user value: route discovery, logging, feedback, and lightweight gamification. Technical feasibility is supported by a standard client–server architecture and commonly used technologies appropriate for iterative development.

2.2.1 SWOT Analysis

Strengths

- Clear niche focus (indoor climbing) with community-driven content.
- Gamification aligned with climbers' intrinsic motivation and progression.
- Actionable, structured feedback loop for route setters and gyms.
- Lightweight tech stack suitable for a student project with iterative delivery.

Weaknesses

- Reliance on user-generated content (requires seeding and moderation).
- Limited resources for content review, partnerships, and support.
- Potential data sparsity if a gym's community adopts slowly (network effects).
- Early analytics are basic; may not meet advanced gym expectations.

Opportunities

- Rapid growth of indoor climbing and new gyms seeking differentiation.
- Partnerships with gyms for events, challenges, and prize sponsorships.
- Extensions to training insights (style profiling, personalized goals).
- Integrations with existing gym systems (check-in, route databases) and social platforms.

Threats

- Competing apps or gyms' own solutions reducing adoption incentives.
- Content/privacy incidents (e.g., inappropriate uploads) harming trust.
- Platform policy changes (app stores) affecting distribution or features.
- Low engagement if gamification is mis-tuned or prizes are inconsistent.

2.3 Comparative Analysis with Existing Solutions

2.3.1 Existing Solutions

A preliminary market scan shows tools that address only parts of the problem: (i) generic training/logbook applications, usually centered on the user and not tied to a gym's live route set; (ii) internal or informal gym route management approaches (e.g., boards/spreadsheets) that lack structured community feedback; and (iii) social platforms that enable sharing, but without a direct connection to routes, setters, seasons, and aggregated gym analytics.

Overall, no direct competitor was identified that offers an end-to-end product combining: gym route management, in-gym route discovery, ascent logging, structured feedback, optional video verification for fair leaderboards, and aggregated style/statistics at both route and gym level.

2.3.2 Benchmarking Analysis

2.4 Innovation Proposal and Added Value

The main innovation lies in merging the climber experience and gym operations into one connected platform. Each ascent can generate value for both sides: climbers receive a richer, more motivating experience (history, progress, community beta, and gamification), while gyms and setters receive structured feedback, difficulty perception signals, and usage trends.

Key added-value points include: (i) an in-gym, scan-to-route flow that reduces friction; (ii) a controlled style taxonomy and crowd-based style characterization that can be aggregated into route and gym profiles; (iii) lightweight gamification (badges, seasons, leaderboards, prizes) to improve retention; and (iv) a data model built to support aggregated statistics without constant expensive recomputation.

2.5 Business Opportunity

The business opportunity is primarily supported by the lack of direct competition identified for a gym-oriented, integrated solution. The product naturally fits a B2B2C positioning:

gyms adopt the platform to strengthen community engagement and improve route lifecycle visibility, while climbers gain a centralized and motivating experience.

A plausible monetization path is subscription/licensing per gym (tiered by feature modules or active users), optionally complemented by premium consumer features. Market entry can start with a small number of pilot gyms to validate adoption, refine the user experience, and then expand to other gyms with similar needs.

2.6 Business Risks

The following risks could affect delivery or adoption of the MVP. Each item includes a primary mitigation.

- **Cold start (adoption/content).** Few early users or videos reduce value. *Mitigation:* seed routes and beta with a pilot gym; launch a short challenge to generate initial content.
- **Content safety.** Inappropriate or IP-infringing uploads harm trust. *Mitigation:* clear guidelines, report/takedown flow, uploader ownership affirmation.
- **Video performance/cost.** Upload/streaming can be slow or exceed free tiers. *Mitigation:* client-side compression, duration/resolution caps, lazy loading, usage monitoring.
- **Scope creep and schedule slip.** Extra features delay MVP. *Mitigation:* milestone feature freeze, time-boxed sprints, “later” backlog.
- **Security & privacy.** Weak auth or poor consent risks data exposure/GDPR issues. *Mitigation:* managed auth, HTTPS, rate limits, least-privilege, consent + delete/export.
- **Third-party dependence.** Changes in auth/storage/CDN pricing or APIs. *Mitigation:* pin versions, thin integration layer, periodic review of usage and costs.

2.7 Constraints

- **Timeline.** Delivery within the Oct 2025–Jun 2026 academic schedule and stated milestones.
- **Budget.** No dedicated funding; rely on free/low-cost tiers and university resources.
- **Team capacity.** Single developer; scope must remain MVP-sized with incremental releases.
- **Scope limits.** No payments, advanced analytics, wearables, or outdoor crag support in the MVP.
- **Compliance.** Must satisfy App Store/Play UGC rules and GDPR (consent, minimal PII, deletion/export).

- **Content/IP and venue rules.** Uploaders must own rights; respect filming policies of partner gyms.

2.8 Resources and Stakeholders

2.8.1 Resources

- **Development team:** Responsible for the overall software development process, including requirements gathering, system design, implementation, integration of features, and maintenance. The development team is also responsible for producing the required documentation.
- **Physical facilities and equipment:** Personal computer, access to computer science labs, and climbing gyms for testing.
- **Software tools:** IDEs, version control (Git), Overleaf for documentation.

2.8.2 Stakeholders

- **Project manager:** Oversees project planning, progress and execution.
- **Climbing gyms:** Provide real-world routes and feedback on system integration.
- **Climbers (end-users):** Use the application, generate content, and provide feedback.
- **Development team:** Student implementing the system, responsible for delivering the product.

3 Specification and Modeling

3.1 Requirements Analysis

3.1.1 User Stories (Illustrative)

- As a climber, I want to scan a tag to open a route so that I can quickly view details and log an attempt.
- As a gym admin, I want to require verification videos for high-grade sends to keep leaderboards fair.
- As a routesetter, I want to view a route's shared feedback, so I can tweak anything needed.

3.1.2 Functional Requirements

The following multi-page table lists functional requirements by category. Each entry has an ID, a Priority Weight (PW: 5=Must, 4=Should, 3=Important, 2=Nice, 1=Future), and a brief description.

Table 3.1: Functional requirements (by category).

ID	PW	Requirement
A. Identity & Accounts		
FR-A1	5	Email/password sign-up, login, logout, and password reset.
FR-A2	4	Profile with editable fields; user can pin up to 3 badges (Free tier limited to 1).
FR-A3	5	Account deletion and data export on request.
B. Gyms & Areas		
FR-B1	5	Gym directory with details and links to each gym's social pages.
FR-B2	2	Sectors/areas metadata and map position inside a gym.
FR-B3	4	QR/NFC station mapping to open route details from tags.
C. Problems/Routes Catalog		
FR-C1	5	Route model with setter identification; user-sourced tags aggregated by consensus.
FR-C2	4	Media attachments: beta videos.
FR-C3	5	Search & filters by grade, style, tags, recency, popularity, sent/unsent, setter.
FR-C4	4	Save routes as Project/Wishlist/Favorite and access from logbook.

ID	PW	Requirement
FR-C5	4	System generates unique route IDs and printable QR/NFC payloads for gyms.
FR-C6	3	Feedback governance: up/down-vote tags and beta tips; low-score/spam hidden.
D. Tick-Logging & Sessions		
FR-D1	5	Quick log: Flash/Send, attempts count, notes, with one-tap from route or scan.
FR-D2	4	Session summary per day: attempts, sends, grade distribution.
FR-D3	4	Offline cache and sync with conflict resolution.
FR-D4	4	Explicit session start and end; compute duration.
FR-D5	2	Auto-suggest session start based on geofence/QR or multiple logs.
E. Progression & Analytics		
FR-E1	4	Personal grade pyramid visualization.
FR-E2	3	Style insights from crowd tags (e.g., slab, overhang, dyno).
FR-E3	3	Personalized route suggestions (Pro).
FR-E4	3	Grade conversion view (Font/V/YDS) respecting gym defaults and user preference.
F. Gamification		
FR-F1	4	Badges/achievements for milestones; user can choose which to display.
FR-F2	4	Streaks (weekly/monthly) with decay and recovery mechanics.
FR-F3	3	Time-boxed challenges/quests.
FR-F4	1	Elo-like ranks (low priority), decay rules applied.
G. Social & Community		
FR-G1	4	Leaderboards by hardest grade, flash rate, volume, and rank (if enabled).
FR-G2	4	Route feedback: beta tips (text), optional beta video, user-supplied tags.
FR-G3	4	Report abuse/spam on users, videos, tags, comments (moderation queue).
FR-G4	4	Gym-configurable video requirement: for selected grade thresholds, a verification video is required for a send to count toward leaderboards; supports admin/judge review or peer threshold; missing video excludes the send from leaderboards until verified.
H. Competitions & Events		
FR-H1	4	Event creation (gym admin): scoring model, eligible routes.

ID	PW	Requirement
FR-H2	4	Athlete score entry: self-entry via QR + verifier workflow or judge entry.
FR-H3	4	Live scoreboard with real-time rankings and exports for screens.
FR-H4	3	Categories (age/gender options), scheduling.
FR-H5	3	Results export (CSV/PDF) for gyms.
I. Route-Setting Workflow (Gym Admin)		
FR-I1	4	Set list/calendar planning; assign setters; workload view.
FR-I2	3	Grade consensus view for setters versus community.
FR-I3	3	Maintenance tickets: report spinning/broken holds; ticket lifecycle.
FR-I4	3	Season reset: archive/rotate sets; close leaderboards per season.
J. Notifications		
FR-J1	4	Push notifications (topics configurable): friends' sends, route retirements, comp updates, rank changes.
L. Monetization & Access Control		
FR-L1	5	Roles and permissions: Climber, Gym Admin, Setter, Judge, Super-Admin (scoped per gym).
FR-L2	4	Gym billing model: €0.50 per active gym-linked user/month.
FR-L3	4	Consumer tiers: Free (1 badge; 5 beta videos/week; no analytics) vs Pro (3 badges; unlimited beta; analytics).
FR-L4	5	Pro subscription purchase flow: native IAP, restore purchases.
FR-L5	3	Trials and promo/coupon redemption for Pro.
O. Admin Web Portal		
FR-O1	4	Web dashboard for gyms: sets in progress, active routes, scan counts.
FR-O2	3	Bulk import/update (FOR REVIEW): consider replacing CSV with inline bulk editor.
FR-O3	3	Audit log: who changed grades/tags/statuses or removed media and when.
P. Data Quality & Mapping		
FR-P1	5	Grade systems: multiple systems supported with internal canonical scale.
FR-P2	4	Crowd-sourced tagging normalized into controlled taxonomy; admin alias/merge.
FR-P3	4	Anti-spoof for QR/NFC: signed payloads with rotation and client verification.

ID	PW	Requirement
Q. Accessibility & Localization (Functional Behaviors)		
FR-Q1	3	Language switching (EN, PT-PT).
FR-Q2	2	Large text mode and larger tap targets.
FR-Q3	2	Color-aware mode (patterns/labels for hold colors).

3.1.3 Non-Functional Requirements

The following nonfunctional requirements define the expected quality attributes of the system.

- **NFR-1 – Performance and Responsiveness.** The mobile app shall load the Home screen and route details within 3 seconds on a stable 4G or Wi-Fi connection. Core actions (scanning a QR, logging a send) must provide visible feedback within 1 second.
- **NFR-2 – Availability.** Cloud services (API and database) shall maintain at least 99% uptime per month, excluding scheduled maintenance.
- **NFR-3 – Offline Operation.** Climbers shall be able to view cached routes and log attempts while offline. The app shall automatically sync new data when a connection is restored.
- **NFR-4 – Data Integrity and Security.** All communication between client and server must use HTTPS (TLS 1.2+). User credentials are encrypted and never stored in plain text. Access control is enforced according to user roles (FR-L1).
- **NFR-5 – Privacy and Data Protection.** The system shall comply with GDPR requirements, allowing users to export or delete their data (FR-A3). Sensitive media such as verification videos must respect privacy settings and consent flags.
- **NFR-6 – Scalability.** The backend shall support up to 10 000 concurrent users without degraded performance, allowing horizontal scaling through containerized deployment.
- **NFR-7 – Maintainability.** The codebase shall follow modular architecture, version control via Git, and continuous integration to enable safe incremental updates.
- **NFR-8 – Usability and Accessibility.** The interface shall follow consistent design guidelines with clear iconography and large tap targets (FR-Q2). The app must remain fully usable on common mobile screens and include a high-contrast or color-aware mode (FR-Q3).

- **NFR-9 – Reliability and Recovery.** In case of service interruption, locally stored session logs must not be lost. Sync operations shall retry automatically until confirmation of success.
- **NFR-10 – Monitoring and Logging.** The backend shall collect anonymized error and performance metrics for issue diagnosis, while excluding personal identifiers.

3.2 Modeling

3.2.1 Data Model (ER)

The data model was designed to support both the climber perspective (ascent logging, feedback, and media) and the gym perspective (route management, seasons/leaderboards, and aggregated analytics). At its core, *users*, *gyms*, and *routes* form the main domain entities. Each route belongs to a gym and stores relevant metadata (grade system/value, color, wall section, setter, and lifecycle status).

Ascent logging is represented by *route_ascents*, which associates a user to a route and stores information such as whether the route was sent, the date, rating, and optional comments. Style characterization is supported by a controlled taxonomy (*style_tags*). Users can select tags per ascent using the association table *ascent_style_tags*. These raw votes can then be aggregated into *route_style_statistics* (per-route) and *gym_style_statistics* (per-gym), enabling trend and profile analysis without recomputing everything in real time.

Motivation and community features are supported by *badges*/*user_badges* and by *leaderboards* with entries and participation snapshots. Optional *prizes* can be linked to leaderboard rankings to encourage consistent participation.

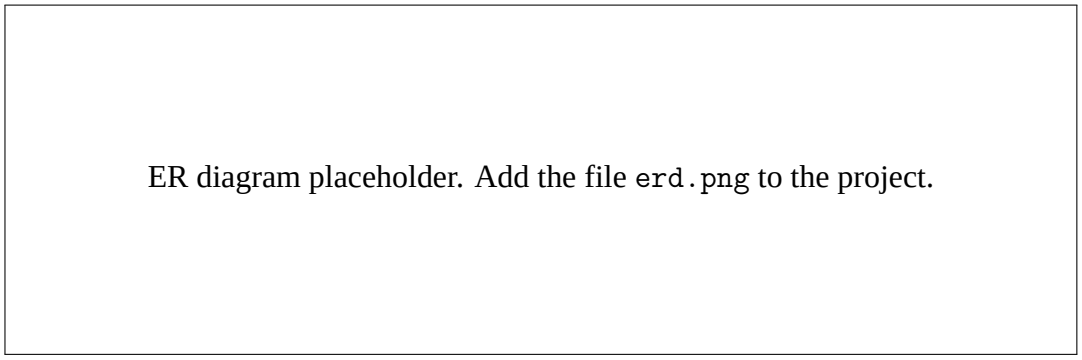


Figure 3.1: Entity–Relationship (ER) diagram of the system.

3.2.2 DB Diagram

Online diagram: dbdiagram.io/d/694574f44bbde0fd74d5a527

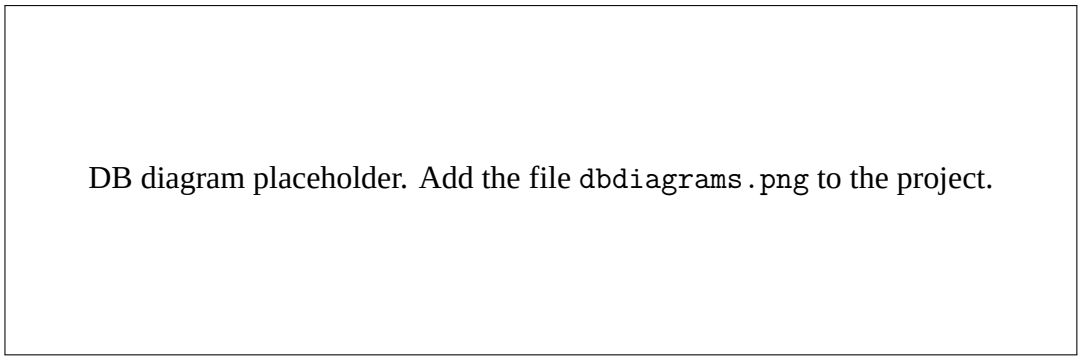
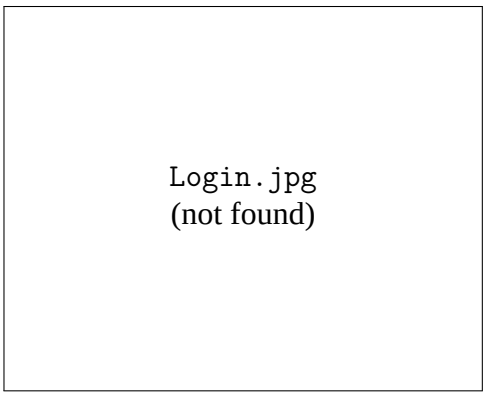


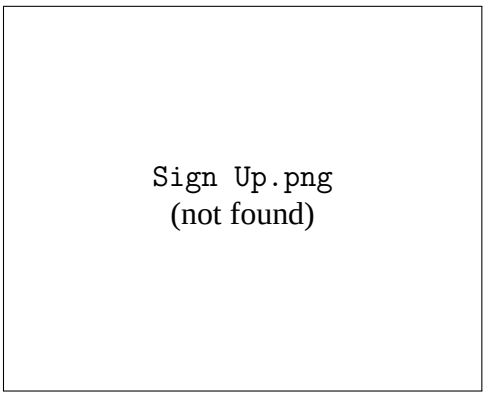
Figure 3.2: Database schema as defined in DBdiagram.

3.3 Interface Prototypes

This section presents the full set of interface mockups produced for the MVP flows.

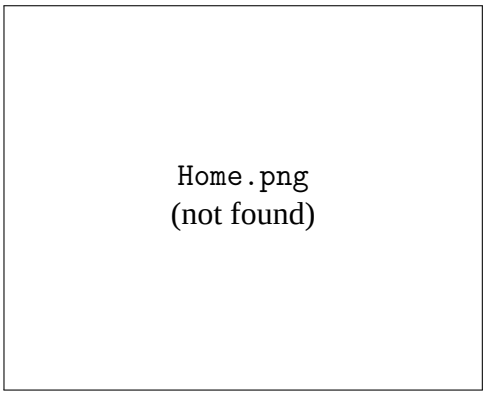


(a) Login screen.

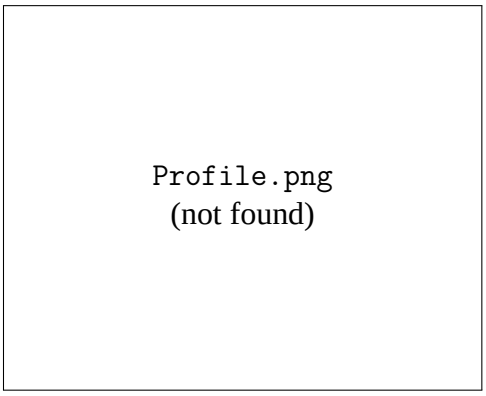


(b) Sign-up screen.

Figure 3.3: Authentication screens.

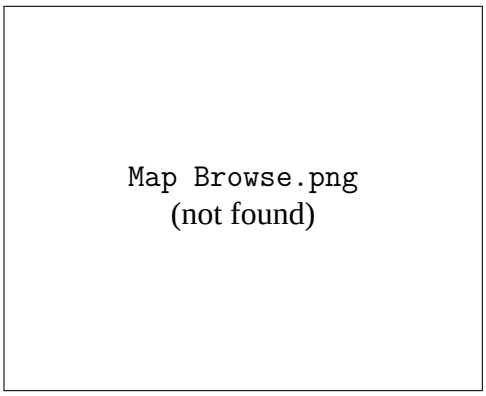


(a) Home screen.

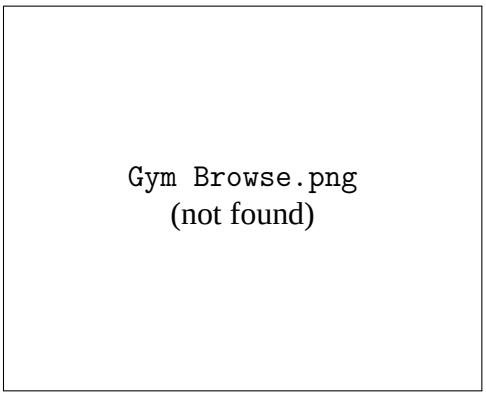


(b) User profile screen.

Figure 3.4: Core screens.

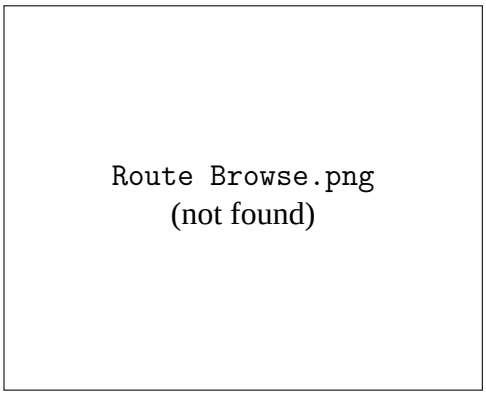


(a) Map browsing screen.

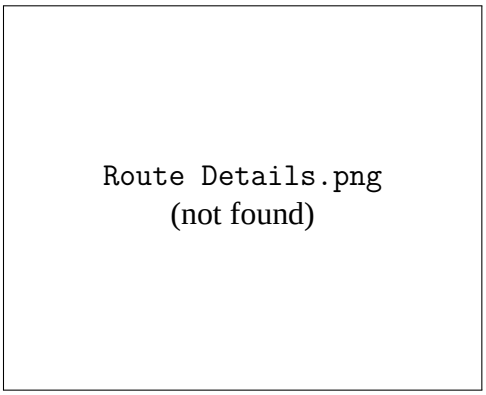


(b) Gym browsing screen.

Figure 3.5: Gym discovery screens.

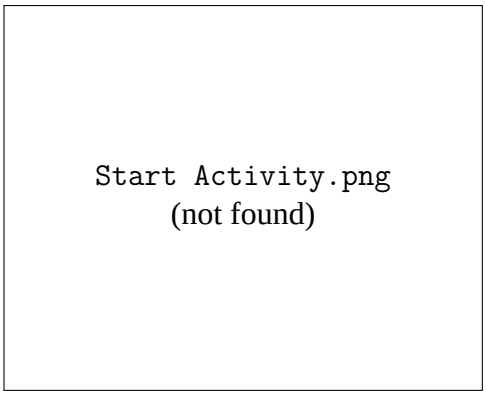


(a) Route browsing and filtering.

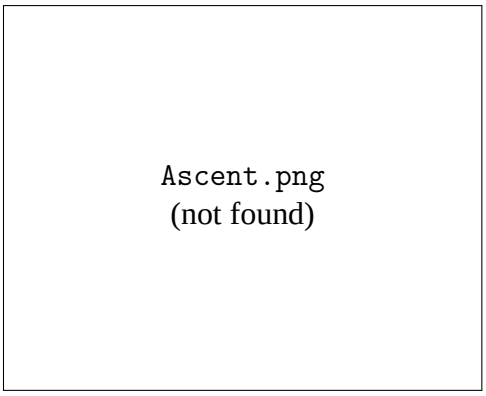


(b) Route details screen.

Figure 3.6: Route browsing screens.

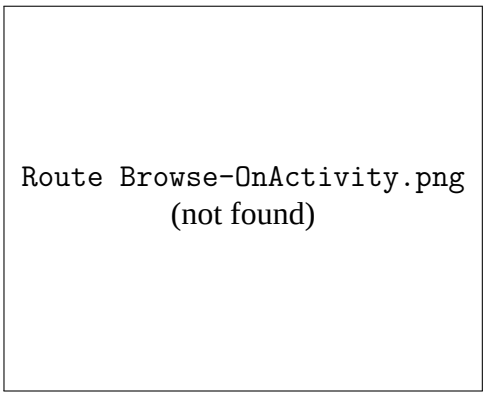


(a) Start activity screen.

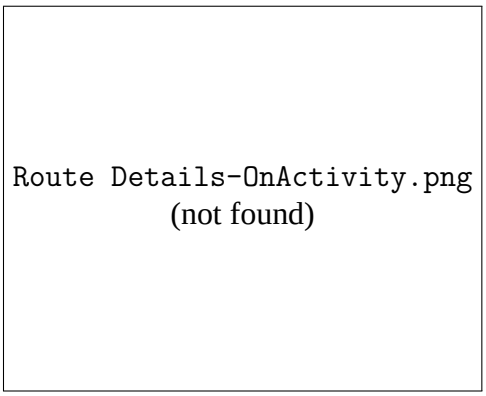


(b) Ascent logging screen.

Figure 3.7: Activity and ascent logging screens.

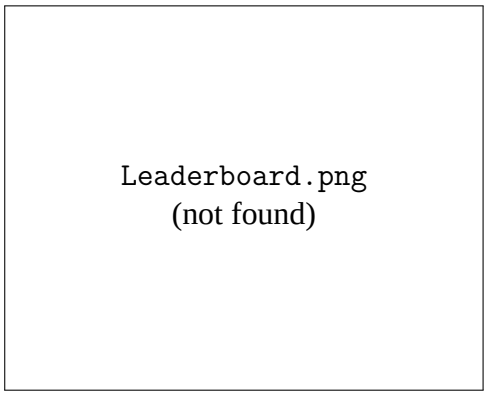


(a) Route browsing while in an activity.



(b) Route details while in an activity.

Figure 3.8: Route screens during an activity.



(a) Leaderboard screen.

Figure 3.9: Leaderboard screen.

4 Developed Solution

4.1 Introduction

The Beta Breaker application has been implemented as a cross-platform mobile application using React Native and Expo, backed entirely by Supabase for database, authentication, storage, and real-time capabilities. As of February 2026, the project has completed 45 development steps across 13 phases, covering all core platform features: authentication, route catalog, session logging, offline support, QR scanning, gamification, social features, notifications, competitions, beta video sharing, and monetization.

The implementation follows a strict Test-Driven Development (TDD) methodology, resulting in 1 027 unit tests and 392 integration tests (1 419 total) across 124 test suites.

4.2 Architecture

The system follows a "Supabase-first, client-heavy" architecture. There is no custom backend server. All API access goes through Supabase PostgREST, Auth, Storage, and Realtime. Authorization is enforced at the database level via Row Level Security (RLS) policies on every table, forming the hard security boundary. Edge Functions (Deno runtime) are used sparingly for operations that cannot run client-side: QR code signing, push notification dispatch, in-app purchase verification, and billing webhooks.

State management is organized into three layers:

1. **Server state (TanStack Query v5):** All data from Supabase (routes, ascents, leaderboards, badges). The database is the source of truth; the client caches and auto-refetches.
2. **Client state (Zustand 5.x):** Ephemeral UI state such as session timer, pending logs, active filters, and bottom sheet visibility. Minimal boilerplate, no providers required.
3. **Local persistence (expo-sqlite):** Offline route cache with 24-hour TTL and an offline action queue. On reconnect, actions are replayed with exponential backoff (1s, 4s, 16s).

The data flow follows four patterns:

- **Reads:** Screen → TanStack Query hook → service layer → PostgREST → Postgres (RLS-filtered).
- **Writes:** User action → Zustand optimistic update → useMutation → service → Postgres triggers (streak/badge/leaderboard) → cache invalidation.
- **Offline:** Action → offlineStore queue (SQLite) → [reconnect] → drain queue → service layer → invalidate caches.

- **Realtime:** Supabase Realtime subscription → Postgres emits changes → TanStack Query cache updated.

4.3 Technologies and Tools Used

Table 4.1 lists the key technologies, their versions, and the justification for each choice.

Table 4.1: Technologies and tools used in the implementation.

Technology	**Version**	**Justification**
React Native	0.81.5	Cross-platform iOS/Android from a single TypeScript codebase.
Expo SDK	54	Managed builds (EAS), OTA updates, and native API wrappers reduce infrastructure complexity.
TypeScript	5.9	End-to-end type safety from database types to UI components.
Supabase	2.95	Postgres + RLS eliminates custom API layer. Includes managed Auth, Storage, Realtime, and Edge Functions.
TanStack Query	5.90	Battle-tested caching, optimistic mutations, and background refetch for all server state.
Zustand	5.0	Minimal boilerplate for client-only state; no providers needed.
NativeWind	4.2	Tailwind utility classes for React Native; dark-first with 25+ color tokens.
expo-sqlite	16.0	Built into Expo SDK; simple SQL interface for offline queue and route cache.
React Hook Form + Zod	7.71 / 4.3	Declarative form validation with TypeScript-first schemas.
expo-camera	17.0	QR code scanning for route identification.
expo-video	3.0	Beta video playback with lazy-loaded thumbnails.

Technology	Version	Justification
react-native-iap	14.7	In-app purchase integration for Pro subscriptions.
expo-notifications	0.32	Push notification delivery and token management.
Jest + RNTL	30.0 / 13.3	Unit testing with React Native Testing Library for component behavior testing.
Supabase CLI	2.75	Local Postgres instance for integration tests against real RLS policies.

4.4 Test and Production Environments

- **Development:** Local machine with `supabase start` (Docker containers for Postgres, Auth, Storage, Realtime). Expo development server with hot reload (`npx expo start`).
- **Testing:** Jest test runner with `jest-expo` preset. Unit tests mock Supabase and native modules. Integration tests run against the local Supabase Postgres instance on port 54322.
- **CI:** GitHub Actions with three stages: lint (ESLint 9+) → type-check (`tsc --noEmit`) → test (`npm test`). Node 24.13.0.
- **Preview builds:** EAS Build with `preview` profile (iOS Simulator + Android APK).
- **Production:** EAS Build with `production` profile for store submission. OTA updates via `eas update` for JavaScript-only changes.

4.5 Scope

The following course subjects are directly applied in the implementation:

- **Databases:** PostgreSQL schema design with 19 tables, foreign keys, indexes, CHECK constraints, triggers, and stored functions. Row Level Security for authorization.
- **Software Engineering:** TDD methodology, requirements traceability, modular architecture, version control (Git), CI/CD pipeline.
- **Programming:** TypeScript, functional programming patterns (pure utility functions), object-oriented patterns (service layer), reactive programming (Realtime subscriptions).
- **Web Technologies:** REST APIs (PostgREST), JWT authentication, HTTPS/TLS, Edge Functions (serverless Deno).
- **Human-Computer Interaction:** Mobile UI design, wireframes, design system (dark-first, color tokens), accessibility considerations.

4.6 Components

The codebase is organized into clearly separated layers, each with a single responsibility.

4.6.1 Database Layer (18 Migrations)

The Supabase PostgreSQL database contains 19 tables organized across 18 sequential migrations:

- **Core tables:** `profiles`, `gyms`, `routes`, `route_ascents`, `style_tags`, `route_style_tags`, `user_gym_roles`.
- **Social tables:** `route_feedback`, `route_feedback_votes`, `route_media`, `content_reports`, `follows`.
- **Gamification tables:** `badges` (19 seeded), `user_badges`, `user_streaks`, `leaderboard_entries`.
- **Competition tables:** `events`, `event_routes`, `event_categories`, `competition_scores`.
- **System tables:** `notifications`, `push_tokens`, `notification_preferences`, `saved_routes`, `audit_log`, `maintenance_tickets`, `seasons`, `subscription_events`.

All tables have RLS enabled with 74+ policies. Seven `SECURITY DEFINER` functions and three triggers handle automated logic (badge awarding, streak updates, leaderboard computation).

4.6.2 Service Layer (19 Services)

Services are thin wrappers around Supabase PostgREST. They contain no business logic, only query construction:

- `auth.service.ts` — Sign-up, sign-in, sign-out, password reset, session management.
- `routes.service.ts` — Route queries with grade, style, and status filters.
- `sessions.service.ts` — Ascent logging, session summaries, session history.
- `leaderboard.service.ts` — Three scoring models (hardest grade, flash rate, volume).
- `media.service.ts` — Beta video upload to Supabase Storage, retrieval, deletion.
- `purchase.service.ts` — Wraps `react-native-iap` for IAP purchase and receipt verification.
- Plus 13 additional services for gyms, profiles, feedback, social, events, scores, challenges, badges, notifications, moderation, saved routes, and subscriptions.

4.6.3 Hook Layer (24 Hooks)

Each hook wraps a service in TanStack Query, providing caching, loading states, error handling, and cache invalidation:

- `useAuth` — Authentication state with `onAuthStateChange` listener and `refreshProfile()`.
- `useRoutes` — Route list query with infinite scroll and filter support.
- `useSubscription` — Purchase and restore mutations with receipt verification.
- `useScoreboardRealtime` — Live competition scoreboard via Supabase Realtime.
- `useOfflineSync` — Offline queue drain with exponential backoff on reconnect.

4.6.4 Store Layer (3 Zustand Stores)

- `sessionStore` — Active climbing session state (timer, pending logs, duration).
- `offlineStore` — Offline action queue backed by expo-sqlite.
- `routeFilterStore` — Active route filter selections (grade, style, wall).

4.6.5 Utility Layer (14 Pure Functions)

Pure TypeScript functions with no side effects, covering: grade conversion (V-scale, Font, YDS with canonical integer mapping 0–30), streak calculation, leaderboard scoring, competition scoring, challenge criteria evaluation, Zod validation schemas, time formatting, ISO week utilities, geolocation, QR code parsing, video validation, entitlement checks, and CSV/PDF export.

4.6.6 Edge Functions (4 Deno Functions)

- `sign-qr` — Signs QR code payloads with EC P-256 keys; admin-authenticated.
- `dispatch-push` — Sends push notifications via Expo Push API with preference checks.
- `verify-iap` — Validates App Store / Play Store receipts, inserts subscription events, updates user tier.
- `billing-webhook` — Handles store renewal, cancellation, and expiration notifications.

4.6.7 UI Components (35+ Components)

Components are organized by domain: `ui/` (8 primitives: Button, Card, Badge, Avatar, TextInput, IconButton, Divider, StarRating), `routes/` (RouteCard, FilterBar, VideoUploadButton, OwnershipModal, BetaVideoPlayer), `session/` (QuickLogSheet, SessionTimer, SessionSummary), `social/` (FeedItem, FeedbackComposer, FollowButton, ReportSheet), `badges/` (BadgePicker, ProfileBadges), `streaks/` (StreakCard, StreakStatusBanner), `challenges/`

(ChallengeCard), `notifications/` (NotificationBell, NotificationItem), `navigation/` (CustomTabBar with FAB), and `subscription/` (Paywall).

4.7 Interfaces

The application uses Expo Router for file-based routing with three route groups:

- `(auth)/` — Unauthenticated screens: Login, Register, Forgot Password.
- `(tabs)/` — Main app with bottom tab bar: Home, Profile, Leaderboards, QR Scanner, Logbook, Map.
- `(admin)/` — Role-gated admin screens: Moderation queue, Event management.

Additional screens include gym detail (`gym/[id]/`), route detail with ascent logging, event scoreboard, user profiles, notification center, and settings.

5 Testing and Validation

5.1 Test Approach

The project follows a strict Test-Driven Development (TDD) methodology where every implementation step begins with failing tests (Red), followed by the minimum code to pass (Green), and then refactoring. This approach ensures that every feature has test coverage from inception and that regressions are caught immediately.

5.2 Test Pyramid

Testing is organized into three levels:

1. **Unit tests (1 027 tests):** Pure utility functions, service query construction, hook behavior, and component rendering. Services mock the Supabase client; hooks mock the service layer; components mock hooks.
2. **Integration tests (392 tests):** Run against a real local PostgreSQL instance via `supabase start`. These tests verify RLS policies, database triggers, stored functions, CASCADE behavior, and UNIQUE constraints. Each test authenticates as a specific user to confirm that row-level security correctly filters data.
3. **End-to-end tests (Phase 20):** Planned for a later phase using Maestro or Detox for full user-flow validation on device.

5.3 Tools and Infrastructure

- **Jest** with `jest-expo` preset — Test runner with TypeScript support via `ts-jest`.
- **React Native Testing Library** — Behavior-driven component testing (render, press, assert text).
- **Local Supabase (Docker)** — Real Postgres with Auth and RLS for integration tests.
- **GitHub Actions CI** — Automated lint $\rightarrow$ type-check $\rightarrow$ test on every pull request.

5.4 Test Coverage by Phase

Table 5.1 shows the test count progression across all 13 completed phases.

Table 5.1: Test counts by development phase.

Phase	Domain	New Tests	Cumulative
0	Scaffolding & CI	3	3
1	Grades, streaks, scoring, validation	49	52
2	Database schema & RLS (integration)	319	371
3	Auth, UI primitives, auth screens	96	467
4	Gyms, routes, tab bar, map	106	573
5	Sessions, QuickLog, logbook	73	646
6	Offline store, route cache, sync	44	690
7	QR scanner, sign-qr Edge Function	23	713
8	Badges, streaks, challenges	158	871
9	Leaderboards, feedback, social	131	1 002
10	Push notifications, notification center	79	1 081
11	Competitions, scoreboard, export	94	1 175
12	Video upload, video playback	79	1 254
13	IAP, entitlements, paywall	39	1 293
Total (unit + integration)			**1 419**

5.5 Risk Analysis

The main testing risks and their mitigations are:

- **Native module mocking:** Libraries like `react-native-iap`, `expo-camera`, and `expo-video` cannot run in a Jest environment. *Mitigation:* comprehensive `jest.mock()` factories in each test file that simulate the native API surface.
- **RLS policy correctness:** Incorrect policies could leak data across users. *Mitigation:* 392 integration tests run against real Postgres, authenticating as different users and verifying that SELECT/INSERT/UPDATE/DELETE are properly scoped.
- **Offline sync conflicts:** Queued actions may conflict with server state. *Mitigation:* last-write-wins strategy with exponential backoff and unit tests covering queue drain, retry, and failure scenarios.

6 Method and Planning

6.1 Development Methodology

The project follows an incremental, test-driven approach organized into 21 sequential phases. Each phase is broken into numbered steps, and each step follows the TDD cycle:

1. **Red:** Write failing tests that define the expected behavior.
2. **Green:** Write the minimum code to make the tests pass.
3. **Refactor:** Clean up duplication, improve naming, re-run all tests.

Version control follows a trunk-based workflow on the `main` branch, with atomic commits per step. Each commit message references the step number (e.g., "feat: add QR scanner screen (Step 7.1)"). A GitHub Actions CI pipeline runs lint, type-check, and the full test suite on every push.

6.2 Milestones

Table 6.1: Project I milestones and target dates.

Event and Deliverables	**Target Date**	**Responsibility**
Project definition and proposal	October 01, 2025	Development team
Project charter approved	October 15, 2025	Development team
System requirements completed	November 05, 2025	Development team
Project plan completed	November 19, 2025[1]	Development team
Lo-fi prototype completed (wireframes and basic interactions)	December 10, 2025	Development team
Hi-fi prototype completed	January 10, 2026[2]	Development team
Prototype implementation (basic functionalities)	March 15, 2026	Development team
Gamification module (achievements + leaderboards) implemented	April 13, 2026	Development team
Testing and feedback round	May 04, 2026	Development team
Final refinements and completion of project report	May 20, 2026	Development team
Final presentation and project closure	June 03, 2026	Development team

[1] 2nd Midterm Presentation: Project Plan and Project Charter completion

[2] 3rd Midterm Presentation: Lo-fi and Hi-fi Prototypes

Gantt chart placeholder. Add the file `gantt.png` to the project.

Figure 6.1: Gantt chart for Project I milestones and schedule.

6.3 Development Progress

As of February 2026, the project has completed 45 steps across 13 of 21 planned phases. Table 6.2 summarizes the completed phases.

Table 6.2: Completed development phases and test counts.

Phase	Description	Tests Added
0	Project Scaffolding & CI Foundation	3
1	Data Foundation (grades, streaks, scoring, validation)	49
2	Database Schema & RLS Policies (19 tables, 74+ policies)	319
3	Authentication & UI Design System (8 components)	96
4	Gyms, Routes & Tab Navigation (11 steps)	106
5	Tick-Logging & Sessions	73
6	Offline Support (SQLite queue, route cache, sync engine)	44
7	QR/NFC Scanning (camera + Edge Function)	23
8	Gamification (badges, streaks, challenges)	158
9	Social & Leaderboards (feedback, follows, moderation)	131
10	Notifications (push tokens, dispatch, in-app center)	79
11	Competitions & Events (CRUD, live scoreboard, export)	94
12	Media / Beta Videos (upload, playback, ownership)	79
13	Monetization (IAP, entitlements, paywall)	39
Total		**1 293**

6.4 Progress Analysis

The implementation is ahead of the original milestone schedule. Core functionalities (authentication, route catalog, session logging) targeted for March 2026 were completed in early February 2026. The gamification module (badges, streaks, challenges, leaderboards) targeted for April 2026 was also completed in February 2026.

Key files produced: 19 service modules, 24 hooks, 35+ components, 14 utility modules, 4 Edge Functions, 18 database migrations, and 3 Zustand stores. The codebase consists of approximately 157 TypeScript source files (excluding tests and generated types).

Remaining phases (14–21) cover: progression analytics, admin web portal, user profile refinements, onboarding, internationalization (EN/PT-PT), monitoring, end-to-end testing, and production deployment.

7 Results

7.1 Test Results

The full test suite executes in approximately 9 seconds and reports:

- **Test suites:** 124 total (123 passing, 1 failing[1]).
- **Unit tests:** 1 027 passing.
- **Integration tests:** 392 passing (run against local Supabase).
- **Total tests:** 1 419.

All core business logic achieves high test coverage: grade conversion (11 tests), streak calculation (8 tests), leaderboard scoring (10 tests), Zod validation schemas (10 tests), entitlement checks (10 tests), and competition scoring (8 tests).

Integration tests verify that all 74+ RLS policies correctly enforce row-level access: users can only read their own data, service-role operations bypass RLS for administrative tasks, and CASCADE deletes propagate correctly across related tables.

7.2 Requirements Compliance

Table 7.1 maps each functional requirement category to its implementation status.

Table 7.1: Requirements compliance summary.

Category	Status	Notes
A. Identity & Accounts	Fulfilled	Email auth, profile, account management, refreshProfile.
B. Gyms & Areas	Fulfilled	Gym directory, QR/NFC mapping, map browse.
C. Routes Catalog	Fulfilled	Route model, filters, search, media, saved routes, style tags.
D. Tick-Logging	Fulfilled	Quick log, session timer/summary, logbook, full ascent form.
E. Analytics	Partial	Grade conversion implemented; pyramid and suggestions deferred (Phase 14).
F. Gamification	Fulfilled	19 badges, streaks with decay/recovery, time-boxed challenges. Elo ranks deferred.

[1]The single failing test (QR roundtrip) requires a local `.env.local` file with Supabase service-role credentials, which is excluded from version control for security reasons.

Category	Status	Notes
G. Social	Fulfilled	Leaderboards (3 models), feedback, follows, activity feed, content reporting.
H. Competitions	Fulfilled	Event CRUD, live scoreboard with Realtime, CSV/PDF export. Score entry deferred.
I. Route-Setting	Partial	Admin screens for route management. Calendar planning and maintenance tickets deferred.
J. Notifications	Fulfilled	Push token registration, dispatch Edge Function, in-app notification center.
L. Monetization	Partial	Pro subscription IAP flow, entitlements, paywall. Trials and gym billing deferred.
O. Admin Portal	Partial	Moderation queue and event management. Web dashboard and audit log deferred.
P. Data Quality	Fulfilled	Grade systems with canonical scale, crowd-sourced tagging, QR anti-spoof (signed JWT).
Q. Accessibility	Not started	Language switching and accessibility modes planned for Phase 18.

Of the 14 requirement categories, 8 are fully fulfilled, 4 are partially fulfilled (with specific items deferred to later phases), and 2 are not yet started. All “Must Have” (PW=5) requirements are implemented.

8 Conclusion

8.1 Conclusion

This report documents the specification and implementation of Beta Breaker, a mobile platform for indoor climbing gyms. The project addressed the identified problem — fragmented route knowledge and lack of structured gym-climber feedback — by building a comprehensive system that connects climbers and gyms through route discovery, ascent logging, beta sharing, and gamification.

The implementation followed a strict TDD methodology across 13 phases and 45 steps, producing 1 419 automated tests (1 027 unit + 392 integration). The system covers all core features required for an MVP: authentication, route catalog, session logging, offline support, QR scanning, gamification (19 badges, streaks, challenges), social features (leaderboards, follows, feedback, moderation), push notifications, competitions with live scoreboards, beta video sharing, and Pro subscription monetization via in-app purchases.

The architecture — Supabase-first with no custom backend, TanStack Query for server state, Zustand for client state, and expo-sqlite for offline persistence — proved effective for a single-developer project, minimizing infrastructure complexity while maintaining security through database-level RLS policies.

8.2 Future Work

The remaining 8 phases (14–21) will complete the platform:

1. **Phase 14 — Progression & Analytics:** Grade pyramid visualization, style insights radar chart, and personalized route suggestions (Pro-gated).
2. **Phase 15 — Admin Web Portal:** Gym dashboard with route management, bulk operations, and audit log.
3. **Phase 16 — Profile & Onboarding:** Profile editing, onboarding flow, and gym selection.
4. **Phase 17 — Score Entry & Verification:** QR-based athlete score entry, judge workflow, and verification flow for competitions.
5. **Phase 18 — Internationalization:** Language switching (EN, PT-PT) and accessibility modes (large text, color-aware holds).
6. **Phase 19 — Polish & Monitoring:** Sentry error tracking, performance monitoring, and UI refinements.
7. **Phase 20 — End-to-End Testing:** Maestro or Detox tests for critical user flows.
8. **Phase 21 — Deployment:** Production EAS builds, store submission, and OTA update pipeline.

Completing these phases will result in a production-ready platform deployable to the App Store and Google Play, ready for pilot testing with partner climbing gyms.

Bibliography

[1] Expo Documentation, *Expo SDK 54 Reference*, 2025. https://docs.expo.dev/

[2] Supabase Documentation, *Supabase Platform Guide*, 2025. https://supabase.com/docs

[3] TanStack, *TanStack Query v5 Documentation*, 2025. https://tanstack.com/query/latest

[4] Meta, *React Native Documentation*, 2025. https://reactnative.dev/docs/getting-started

[5] Zustand Contributors, *Zustand State Management Documentation*, 2025. https://zustand.docs.pmnd.rs/

[6] NativeWind Contributors, *NativeWind v4 Documentation*, 2025. https://www.nativewind.dev/

[7] PostgreSQL Global Development Group, *PostgreSQL 15 Documentation — Row Security Policies*, 2025. https://www.postgresql.org/docs/15/ddl-rowsecurity.html

[8] Jest Contributors, *Jest Testing Framework Documentation*, 2025. https://jestjs.io/docs/getting-started

[9] react-native-iap Contributors, *react-native-iap v14+ Documentation*, 2025. https://react-native-iap.dooboolab.com/